Plan de Migración — App Móvil TTPN

Fecha: 2026-04-10
Dirigido a: Desarrollador móvil Android
Contexto técnico completo: `APP_MOVIL_BE_AJUSTES.md`, `TRAVEL_COUNTS_ANALISIS.md`

Resumen ejecutivo

La app Android actualmente se comunica con un servidor PHP (`ttpnplaystore-a94805096f88.herokuapp.com`) que actúa como proxy delgado sobre la base de datos PostgreSQL en Supabase. Ruby on Rails es y siempre ha sido el backend real: creó todas las tablas, triggers y funciones PostgreSQL. La migración elimina el proxy PHP en dos fases:

Fase	Qué cambia	Riesgo	Duración estimada
Fase 1	La app apunta a Supabase directamente (o PHP levantado contra Supabase)	Bajo — sin cambios de lógica	1–2 días
Fase 2	La app usa la REST API de Rails (JWT, endpoints RESTful)	Medio — requiere refactorizar auth y llamadas HTTP	2–3 semanas

PHP desaparece al finalizar Fase 2.

FASE 1 — Conexión directa a Supabase (puente temporal)

Objetivo

Que la app siga funcionando mientras PHP apunta a la base de datos correcta (Supabase en lugar del servidor legacy). Sin cambios de lógica en la app.

Contexto

PHP ya tiene el SQL correcto y llama las funciones PostgreSQL (`buscar_booking`, `buscar_nomina`, etc.) que existen en Supabase. Solo necesita configurarse para apuntar ahí.

Checklist Fase 1

Tarea 1.1 — Verificar conexión PHP → Supabase

Responsable: BE / DevOps
Estimado: 2–4 horas
Estado: Pendiente BE

- ☐ Configurar `ttpn_php/config/db.php` (o equivalente) con la URL de Supabase:

```
Host:     db.cvuxaldvwgttpknmfjgo.supabase.co
Port:     6543
Database: postgres
User:     postgres
Password: (ver .env del proyecto — variable DATABASE_PASSWORD)
```

- [ ] Verificar que todos los PHP endpoints responden correctamente
- [ ] Probar `SELECT_MAX_TRAVEL_COUNT.php`, `Gasto_INSERT_TRAVEL_COUNTS.php` con datos reales

Tarea 1.2 — Validar app contra nuevo servidor PHP

Responsable: Desarrollador móvil
Estimado: 2–4 horas

- [ ] Confirmar que `Constants.java` apunta al servidor PHP correcto:

```java
// Archivo:
app/src/main/java/com/miTTPN/gascontrol/Utilidades/Constants.java
public static final String BASE_URL = "https://ttpnplaystore-
a94805096f88.herokuapp.com/";
// Cambiar SOLO si el servidor PHP se mueve a nueva URL
```

- [ ] Hacer login con credencial de prueba y confirmar sesión
- [ ] Crear un travel count de prueba y confirmar que aparece en Supabase
- [ ] Verificar que `viaje_encontrado`, `ttpn_booking_id` y `payroll_id` se asignan correctamente post-INSERT

Tarea 1.3 — Smoke test de flujos críticos

Responsable: Desarrollador móvil
Estimado: 3–4 horas

- [ ] Login / Logout
- [ ] Listado de bookings del chofer
- [ ] Creación de travel count (Entrada y Salida)
- [ ] Listado de gasolineras y carga de gasolina
- [ ] Check de vehículo (chofer, entrega, recepción)

Criterio de éxito Fase 1: App funciona igual que antes, pero los datos viven en Supabase.

FASE 2 — Migración a REST API de Rails

Objetivo

Eliminar PHP completamente. La app se comunica directamente con el servidor Rails usando JWT (tokens de sesión) y endpoints RESTful estándar.

Cambio arquitectónico

```
ANTES:  App Android → PHP (Heroku) → PostgreSQL (Supabase)
DESPUÉS: App Android → Rails API (Heroku) → PostgreSQL (Supabase)
```

Requisitos previos (responsabilidad BE/FE — ver §4)

Antes de que el desarrollador móvil comience Fase 2, el backend debe entregar:

1. ☐ Endpoint de login JWT funcionando (`POST /api/v1/auth/login`)
2. ☐ Endpoints de travel_counts completos (CREATE, INDEX, SHOW)
3. ☐ `buscar_nomina()` agregada al trigger de PostgreSQL
4. ☐ Override de `ttpn_foreign_destiny_id` resuelto en backend
5. ☐ Ambiente de staging disponible para pruebas

Checklist Fase 2 — Desarrollador móvil

Bloque A — Autenticación (1–2 días)

A.1 — Reemplazar login PHP por JWT Rails

Archivo actual: `ActivitiesChofer/Login/` (búsqueda de `LOGIN.php` o similar)
Estimado: 4–6 horas

- ☐ Cambiar `BASE_URL` en `Constants.java`:

  ```
  // ANTES:
  public static final String BASE_URL = "https://ttpnplaystore-
  a94805096f88.herokuapp.com/";
  // DESPUÉS (URL de Rails — confirmar con BE):
  public static final String BASE_URL = "https://<rails-api-host>/";
  ```

- ☐ Cambiar el endpoint de login:

  ```
  // ANTES: POST /ttpn_php/login.php
  // DESPUÉS: POST /api/v1/auth/login
  // Body (JSON, sin wrapper):
  // { "email": "usuario@ttpn.com.mx", "password": "contraseña" }
  ```

- ☐ Cambiar lectura de respuesta de login:

```
// ANTES: PHP devolvía campos sueltos (employee_id, vehicle_id, etc.)
// DESPUÉS: Rails devuelve:
// {
//   "token": "eyJ...",           ← guardar en SharedPreferences
//   "user": {
//     "id": 10,
//     "email": "...",
//     "employee": {
//       "id": 10,
//       "nombre": "...",
//       "vehicle_id": 5
//     }
//   }
// }
```

- [ ] Guardar `token` en SharedPreferences (reemplaza la sesión actual):

```
SharedPreferences.Editor editor = prefs.edit();
editor.putString("jwt_token", response.getString("token"));
editor.putInt("employee_id", employee.getInt("id"));
editor.putInt("vehicle_id", employee.getInt("vehicle_id"));
editor.apply();
```

- [ ] Agregar header Authorization a TODAS las llamadas HTTP:

```
// En AndroidNetworking o Retrofit, agregar interceptor:
.addHeaders("Authorization", "Bearer " + jwtToken)
```

- [ ] Manejar respuesta 401 (token expirado) → redirigir a login

A.2 — Verificar logout

Estimado: 1 hora

- [ ] `POST /api/v1/auth/logout` con header Authorization
- [ ] Limpiar SharedPreferences al hacer logout

Bloque B — Travel Counts (3–5 días)

Este es el módulo más crítico. Ver `TRAVEL_COUNTS_ANALISIS.md` para análisis completo.

B.1 — Eliminar fetch de max_id

Estimado: 2 horas
Archivo: `ConteoViajes.java` — buscar `SELECT_MAX_TRAVEL_COUNT.php`

- [ ] Eliminar la llamada GET que obtiene el ID máximo antes de insertar:

```
// ELIMINAR este patrón:
// GET /ttpn_php/SELECT_MAX_TRAVEL_COUNT.php
// sTravel_count_id = maxId + 1
```

- [ ] El ID lo asigna automáticamente PostgreSQL. Leer el `id` de la respuesta POST:

```
int newId = response.getInt("id");
```

B.2 — Cambiar endpoint de creación

Estimado: 3–4 horas
Archivo: `ConteoViajes.java` — buscar `Gasto_INSERT_TRAVEL_COUNTS.php`

- [ ] Cambiar de `POST /ttpn_php/Gasto_INSERT_TRAVEL_COUNTS.php` a:

```
// DESPUÉS: POST /api/v1/travel_counts
// Content-Type: application/json
// Authorization: Bearer <token>
// Body:
// {
//   "travel_count": {
//     "employee_id": 10,
//     "vehicle_id": 5,
//     "client_branch_office_id": 42,
//     "ttpn_service_type_id": 1,
//     "fecha": "2026-04-10",
//     "hora": "08:30:00",
//     "costo": 250.00
//     // NO enviar: id, status, created_at, updated_at
//     // OPCIONAL: ttpn_foreign_destiny_id (el BE lo resuelve si
falta)
//   }
// }
```

- [ ] Leer respuesta 201 y guardar `id`, `clv_servicio`, `viaje_encontrado`, `ttpn_booking_id`:

```
// Respuesta exitosa:
// {
//   "id": 1235,
//   "clv_servicio": "42-2026-04-10-08:30:00-1-2-5",
//   "viaje_encontrado": true,
//   "ttpn_booking_id": 890,
//   "payroll_id": 12,
```

```
//   ...
// }
```

- [ ] Manejar errores 422 (validación) y 401 (auth)

B.3 — Eliminar override hardcodeado de destino

Estimado: 1 hora
Archivo: `ConteoViajes.java` líneas ~1249-1283

- [ ] **Eliminar** este bloque de código (el BE resuelve el destino):

```
// ELIMINAR:
if (sTtpn_foreign_destiny_id.equals("-1")) {
    sTtpn_foreign_destiny_id = "1";
}
if (sClient_branch_office_id.equals("263") || ...) {
    sTtpn_foreign_destiny_id = "11";
}
```

- [ ] Si el spinner de destino retorna -1 (ninguno seleccionado), simplemente no enviar `ttpn_foreign_destiny_id` en el body — el BE lo resuelve.

Nota: Esto requiere que BE haya completado el Gap 3 (campo `default_ttpn_foreign_destiny_id` en `client_branch_offices`).

B.4 — Listado de travel counts

Estimado: 2–3 horas

- [ ] `GET /api/v1/travel_counts?employee_id=10&fecha=2026-04-10`
- [ ] Adaptar el parser de respuesta al formato JSON (vs texto/CSV del PHP actual)

Bloque C — Bookings (2–3 días)

C.1 — Listado de bookings del chofer

Estimado: 3–4 horas
PHP actual: `SELECT_BOOKINGS_CHOFER.php` o similar

- [ ] `GET /api/v1/ttpn_bookings?employee_id=10&fecha=2026-04-10`
- [ ] Adaptar UI a respuesta JSON

C.2 — Detalle de booking

- [ ] `GET /api/v1/ttpn_bookings/:id`

Bloque D — Gasolina (1–2 días)

D.1 — Carga de gasolina

Estimado: 3–4 horas

- [ ] Eliminar fetch de max_id (mismo patrón que travel_counts)
- [ ] `POST /api/v1/gas_charges`
- [ ] Leer `id` de la respuesta

D.2 — Gasolineras disponibles

- [ ] `GET /api/v1/gas_stations`

Bloque E — Checks de vehículo (2–3 días)

Hay 4 tipos de check: chofer, entrega, recepción, auditoría.

E.1 — Crear check

Estimado: 4–6 horas (todos los tipos)

- [ ] `POST /api/v1/vehicle_checks` con `{ "check_type": "chofer", ... }`
- [ ] Confirmar con BE el campo `check_type` o si usa tipos separados

E.2 — Listado de checks

- [ ] `GET /api/v1/vehicle_checks?vehicle_id=5`

Bloque F — Otros módulos (3–5 días)

Módulo	Endpoint Rails	PHP actual
Rutas fijas	`GET /api/v1/fixed_routes`	`SELECT_FIXED_ROUTES.php`
Puntos de ruta	`GET /api/v1/crdhr_points`	`SELECT_CRD_POINTS.php`
Días de ruta	`GET /api/v1/cr_days`	`SELECT_CR_DAYS.php`
Notificaciones FCM	`PUT /api/v1/users/:id` (firebase_token)	`UPDATE_FIREBASE_TOKEN.php`
Clientes-empleados	`GET /api/v1/client_employees`	`SELECT_CLIENT_EMPLOYEES.php`
Broadcast	`GET /api/v1/broadcast_receivers`	`SELECT_BROADCAST.php`

Bloque G — Manejo de sesión y errores globales (1 día)

G.1 — Interceptor HTTP global

Estimado: 3–4 horas

- ☐ Crear interceptor (OkHttp o AndroidNetworking) que:
  - Agrega `Authorization: Bearer <token>` a toda request
  - En respuesta 401 → limpiar sesión y redirigir a pantalla de login
  - En respuesta 422 → mostrar errores del campo `errors` en el JSON
  - En respuesta 5xx → mostrar mensaje genérico de error de servidor

G.2 — Refresh de token (si aplica)

- ☐ Preguntar a BE si el JWT tiene expiración y si hay endpoint de refresh
- ☐ Si no hay refresh, al expirar el token redirigir a login

Resumen de tiempos estimados — Fase 2

Bloque	Módulo	Estimado
A	Autenticación JWT	1–2 días
B	Travel Counts	3–5 días
C	Bookings	2–3 días
D	Gasolina	1–2 días
E	Vehicle Checks	2–3 días
F	Otros módulos	3–5 días
G	Infraestructura HTTP	1 día
Total		**13–21 días hábiles**

---

4. Tareas pendientes en BE (bloqueantes para Fase 2)

Estas tareas son responsabilidad del equipo de backend Rails. El desarrollador móvil no puede avanzar en los módulos relacionados hasta que estén listas.

🔴 Crítico — sin esto los datos quedan incorrectos

BE-1 — Agregar `buscar_nomina()` al trigger PostgreSQL

Estado: ✅ Migración creada — **pendiente correr `db:migrate`**
Archivo: `ttpngas/db/migrate/20260410000002_add_buscar_nomina_to_travel_counts_trigger.rb`
Problema: El trigger `sp_tctb_insert` no asigna `payroll_id`. La función `buscar_nomina()` ya existe en Supabase — solo faltaba invocarla.
Fix aplicado en migración:

```
IF NEW.payroll_id IS NULL THEN
  NEW.payroll_id := (SELECT buscar_nomina());
END IF;
```

Comando para aplicar:

```
docker compose exec api bundle exec rails db:migrate
```

BE-2 — Resolver discrepancias post-INSERT en TravelCount

Archivo: `ttpngas/app/models/travel_count.rb`
Problema: PHP hace un UPDATE a `discrepancies` después de cada INSERT exitoso. Ruby no lo hace.
Fix:

```
after_create :resolver_discrepancia_pnc

def resolver_discrepancia_pnc
  return unless ttpn_booking_id.present?
  Discrepancy.where(kpi: 'pnc', record_id: ttpn_booking_id, status: true)
             .update_all(status: false)
end
```

BE-3 — Override de destino por sucursal en backend

Archivo: `ttpngas/app/controllers/api/v1/travel_counts_controller.rb`
Problema: La lógica que mapea sucursales especiales (263, 262, 261, 196) → destino 11 está hardcodeada en la app Android. Si se elimina de la app, el backend debe resolverlo.
Fix opción A — Campo en tabla:

```
rails g migration AddDefaultForeignDestinyToClientBranchOffices
default_ttpn_foreign_destiny_id:integer
```

```
UPDATE client_branch_offices SET default_ttpn_foreign_destiny_id = 11
WHERE id IN (263, 262, 261, 196);
```

Fix opción B — En el controller:

```
def resolve_foreign_destiny
  return if params.dig(:travel_count, :ttpn_foreign_destiny_id).present?
  branch = ClientBranchOffice.find(params.dig(:travel_count,
```

```
  :client_branch_office_id))
    params[:travel_count][:ttpn_foreign_destiny_id] =
      branch.default_ttpn_foreign_destiny_id ||
      TtpnForeignDestiny.find_by(nombre: 'Chihuahua')&.id
  end
```

🟡 Importante — necesario antes de deprecar PHP

BE-4 — Pasar `created_by_id` desde current_user

Archivo: `ttpngas/app/controllers/api/v1/travel_counts_controller.rb`

```
@travel_count.created_by_id = current_user.id
@travel_count.updated_by_id = current_user.id
```

BE-5 — Controladores faltantes

Los siguientes endpoints no existen aún en Rails (ver `APP_MOVIL_BE_AJUSTES.md`):

- `CooTravelRequestsController` (solicitudes de viaje COO)
- `CooTravelEmployeeRequestsController`
- `CrdHrsController` (horas de ruta)
- `CrdhRoutesController`
- `CrdhrPointsController` (puntos de revisión en ruta)
- `CrDaysController`
- `BroadcastReceiversController`
- `ClientEmployeesController`
- `FixedRoutesController`

BE-6 — Acciones faltantes en controladores existentes

- `UsersController` → acción para actualizar `firebase_token`
- `VehicleChecksController` → acciones separadas por tipo (chofer/entrega/recepción/auditoría)
- `TravelCountsController` → acciones `authorize` y `reject`

---

5. Tareas pendientes en FE (módulo Acceso API)

El frontend Quasar debe tener el módulo de Acceso API funcional para que la app móvil opere sin PHP. Esto es independiente del trabajo del desarrollador móvil pero es parte del plan global.

Tarea FE	Descripción	Prioridad
FE-1	Vista de monitoreo de requests API (logs de llamadas)	🟡
FE-2	Gestión de tokens JWT por usuario	🔴
FE-3	Panel de configuración de permisos de API (qué puede hacer cada rol)	🟡

Tarea FE	Descripción	Prioridad
FE-4	Visualización de travel_counts creados desde la app	🟡
FE-5	Alertas cuando `viaje_encontrado = false`	🟡

---

6. Diagrama de dependencias

```
Fase 1:
  [PHP ajustado a Supabase] ──► [App Android valida] ──► ✅ Deploy

Fase 2:
  [BE-1 buscar_nomina]     ──┐
  [BE-2 discrepancias]     ──┤
  [BE-3 override destino] ──┼──► [App Bloque B – Travel Counts] ──► ✅
  [BE-4 created_by_id]     ──┘

  [Login JWT Rails]        ──► [App Bloque A – Auth] ──► desbloquea todo
lo demás

  [BE-5 controladores]     ──► [App Bloques C-F] ──► ✅ PHP deprecado
```

---

7. Entregables de cada fase

Al finalizar Fase 1

- ☐ App funciona con datos en Supabase
- ☐ No hay errores de conexión ni de datos en flujos críticos

Al finalizar Fase 2

- ☐ `Constants.java` apunta a Rails API
- ☐ PHP ya no recibe tráfico
- ☐ Todos los flujos de la app pasan por endpoints RESTful
- ☐ Los registros de `travel_counts` tienen `payroll_id` correcto
- ☐ Las discrepancias `pnc` se resuelven automáticamente
- ☐ El destino por sucursal lo resuelve el backend

---

8. Contacto y referencias

Recurso	Ubicación
Análisis travel_counts	`TRAVEL_COUNTS_ANALISIS.md`
Mapeo completo PHP → Rails	`APP_MOVIL_BE_AJUSTES.md`
Tareas pendientes generales	`PENDIENTES.md`

Recurso	Ubicación
URL Rails API (staging)	Confirmar con BE
Credenciales Supabase	Ver `.env` del proyecto (variable DATABASE_URL)
Credenciales de prueba	`admin@ttpn.com.mx` / `Ttpn8869*`